

PERTEMUAN 5

VIEWING DAN CLIPPING 2D

Kemampuan Akhir yang Diharapkan (KAD):

Mahasiswa mampu memahami dan mengimplementasikan konsep transformasi *windows-viewport* dan clipping objek 2D.

Pokok Bahasan:

1. Konsep *view* 2D
2. Proses transformasi *windows-viewport* serta komputasinya
3. Proses *clipping* garis dengan algoritma-algoritma standar
4. Implementasi clipping garis pada program dengan Java.
5. Soal Latihan

5.1 Konsep View 2D

Biasanya, paket aplikasi grafik menyediakan fasilitas bagi *user* untuk memilih bagian gambar yang ditampilkan di monitor, juga bagian gambar yang tidak ditampilkan di monitor. Sistem koordinat kartesian sebagai kerangka acuan koordinat dunia bisa digunakan untuk mendefinisikan gambar. Untuk gambar 2D yang ditampilkan di layar monitor, bisa berisi keseluruhan gambar atau hanya sebagian saja dari gambar tersebut. *User* bisa memilih satu area saja untuk ditampilkan di layar, atau memilih beberapa area untuk ditampilkan secara bersama-sama di layar monitor, atau bisa digunakan untuk kebutuhan tampilan animasi. Bagian gambar yang terpilih kemudian dipetakan ke *device coordinate*. Transformasi dari *world coordinates system* ke *device coordinates system* sebenarnya melibatkan beberapa operasi geometri yaitu translasi, rotasi dan scaling maupun prosedur-prosedur untuk menghapus bagian gambar yang tidak ditampilkan di layar (*clipping*) karena bagian tersebut memang berada di luar pandangan.

Dalam komputer grafik ada 5 macam sistem koordinat kartesian, yaitu:

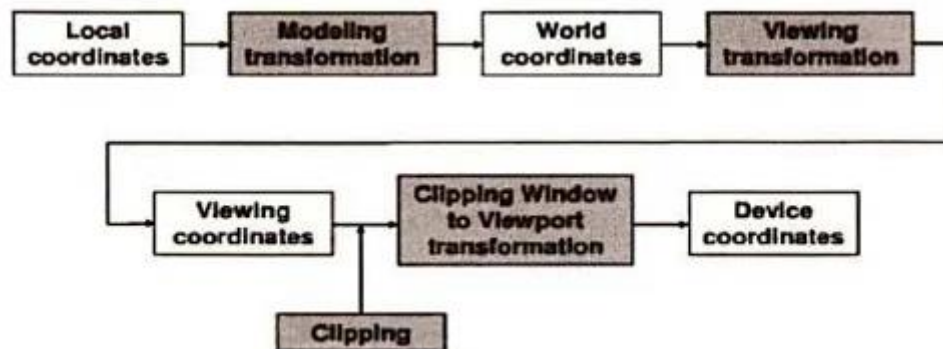
- a. *Modeling coordinate/local coordinate system*,
- b. *World coordinate system*,
- c. *Viewing coordinate system*,
- d. *Normalized device coordinate system*,
- e. *Device coordinate system*.

Modeling coordinate system adalah koordinat tempat objek grafik dibuat secara sendiri-sendiri. Hal ini memungkinkan kita untuk membangun bentuk objek secara terpisah dalam ruang yang mempunyai kerangka acuan yang berbeda-beda. Bila objek-objek sudah terbentuk, kemudian objek-objek tadi kita susun (ditempatkan) pada suatu tempat yang posisi antara satu objek dengan objek yang lain relatif berbeda terhadap suatu kerangka acuan, maka kerangka acuan ini disebut *World Coordinate System*. Penempatan objek tersebut biasanya melibatkan proses translasi, rotasi, bahkan proses rotasi dan translasi sekaligus.

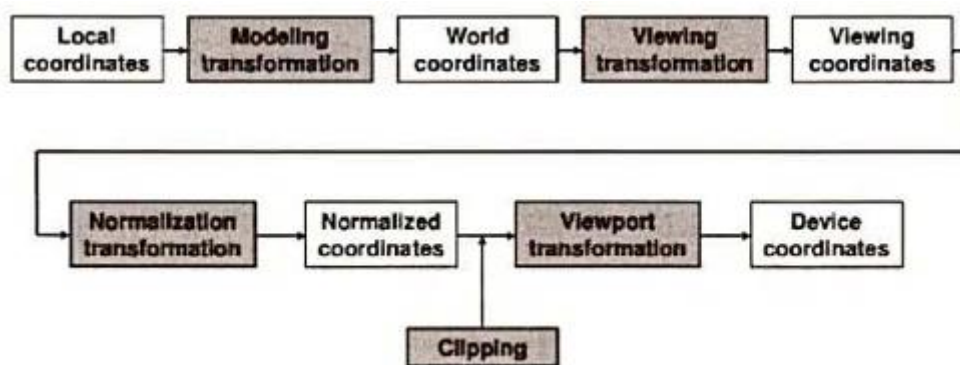
Viewing coordinate system adalah koordinat objek relatif terhadap kerangka acuan pengamat. *Normalized device coordinate* adalah normalisasi koordinat objek yang terletak pada *viewing coordinate*. Nilai koordinat dinormalisasi antara -1 dan 1. Tetapi ada juga sistem grafik lain yang menggunakan nilai di antara 0 dan 1.

Device Coordinate adalah sistem koordinat bidang fisik dimana bisa berupa bidang monitor, titik pada printer laser, atau mm pada *plotter*. *Viewing 2D* adalah urutan transformasi sistem pandang yang dimulai dari *Local Coordinates System*, dilanjutkan ke *World Coordinates*

System sampai pada *Device Coordinates System*. Gambar 5.1 menunjukkan urutan proses viewing 2D, (a) *Viewing 2D* tanpa menggunakan proses normalisasi koordinat, (b) *Viewing 2D* menggunakan proses normalisasi koordinat.



Gambar 5.1 (a)

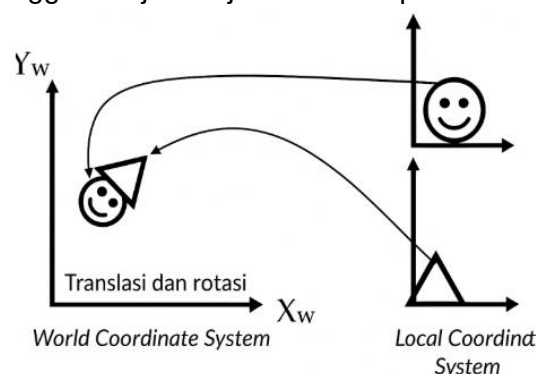


Gambar 5.1 (b)

Gambar 5.1 Urutan viewing 2D. (a) tanpa proses normalisasi koordinat, (b) menggunakan proses normalisasi koordinat

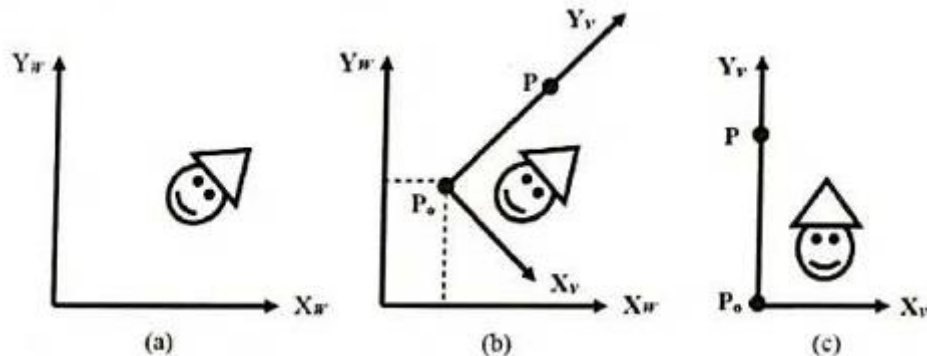
5.1.1 Modeling Transformation

Modeling Transformation adalah proses transformasi model-model obyek dari *local coordinates system* ke *World Coordinates System*. Proses ini biasanya melibatkan transformasi geometri seperti translasi, rotasi, skala dan lain-lain. Perhatikan Gambar 5.2. Model-model objek lingkaran dan segitiga semula digambar pada *Local Coordinate System*. Koordinat tiap-tiap obyek mempunyai kerangka acuan sendiri-sendiri yang semuanya mempunyai koordinat titik asal (0,0). Melalui transformasi geometri, semua objek dipindah ke *World Coordinates System* hingga menjadi objek baru berupa boneka.

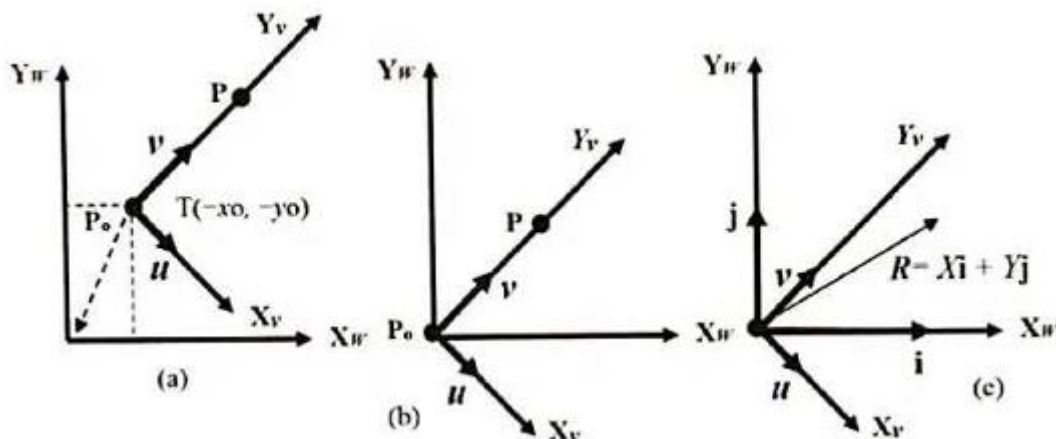
Gambar 5.2 Transformasi objek dari *local coordinate system* ke *world coordinate system*

5.1.2 Viewing Transformation

Gambar 5-3 menunjukkan *Viewing Transformation*. Gambar 5-3(a) menunjukkan sebuah objek berada pada *World Coordinate System* yang mempunyai arah pandang atas Y_w . Bila kita ingin melihat boneka dari titik P_o dengan arah pandang atas $P-P_o$, arah pandang baru ini disebut *Viewing Coordinates System* dengan arah pandang atas Y_v seperti yang ditunjukkan pada Gambar 5-3(b). Karena itu untuk melihat gambar objek dari sisi pengamat yang terletak pada titik P_o dengan arah pandang atas Y_v , hingga tampak seperti yang ditunjukkan pada Gambar 5-3(c), diperlukan transformasi koordinat dari *World Coordinates System* ke *Viewing Coordinates System* yang disebut sebagai *Viewing Transformation*.



Gambar 5.3 Transformasi dari *World Coordinates System* ke *Viewing Coordinates System*



Gambar 5.4 langkah-langkah perhitungan untuk *viewing transformation*

Perhatikan titik $P(x, y)$ dan $P_o(x_o, y_o)$ yang bertindak sebagai vektor arah pandang atas bagi pengamat (viewer) atau sebagai sumbu Y_v dari *Viewing Coordinates System*. Vektor satuan v pada arah Y_v pada *Viewing Coordinates System* adalah:

$$v = \frac{P - P_o}{|P - P_o|} = \frac{(x - x_o, y - y_o)}{\sqrt{(x - x_o)^2 + (y - y_o)^2}} = (v_x, v_y)$$

Vektor satuan u pada arah X_v pada *Viewing Coordinates System* dihitung dengan cara perkalian silang antara vektor satuan v dengan sumbu z pada *Viewing Coordinates System*, yaitu:

$$u = v \times (0, 0, 1) \text{ atau } u = (v_y, -v_x)$$

Pertama sumbu koordinat *Viewing Coordinates System* ditranslasikan ke pusat koordinat *World Coordinates System* seperti pada Gambar 5-4(a), hasilnya adalah Gambar 5-4(b). Proses ini bisa diwakili oleh matriks translasi berikut,

$$T = \begin{bmatrix} 1 & 0 & -x_0 \\ 0 & 1 & -y_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ 1 \end{bmatrix}$$

Pada Gambar 5-4(c) berlaku perhitungan berikut:

- Vektor satuan dalam arah sumbu-XY masing-masing adalah i dan j .
- Vektor satuan dalam arah sumbu $U-V$ masing-masing adalah u dan v .
- Vektor posisi R dilihat dari sumbu-XY adalah:

$$R = x_w \cdot i + y_w \cdot j$$

- Vektor posisi U dilihat dari sumbu-XY adalah:

$$U = u_x \cdot i + u_y \cdot j$$

- Vektor posisi V dilihat dari sumbu-XY adalah:

$$V = v_x \cdot i + v_y \cdot j$$

Maka, vektor posisi R dilihat dari sumbu-UV adalah:

$$R = (R \cdot U) \cdot u + (R \cdot V) \cdot v$$

$$R = (x_w \cdot u_x + y_w \cdot u_y) \cdot u + (x_w \cdot v_x + y_w \cdot v_y) \cdot v$$

Dalam hal ini vektor posisi R dilihat dari sumbu-UV bisa diwakili oleh matriks berikut:

$$R = \begin{bmatrix} u_x & u_y & 0 \\ v_x & v_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ 1 \end{bmatrix}$$

Sehingga matriks transformasi M dari *World Coordinates System* ke *Viewing Coordinates System* dinyatakan sebagai komposisi matriks berikut:

$$M = R * T(-x_0, -y_0)$$

$$M = \begin{bmatrix} u_x & u_y & 0 \\ v_x & v_y & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -x_0 \\ 0 & 1 & -y_0 \\ 0 & 0 & 1 \end{bmatrix}$$

Sehingga posisi titik terhadap *Viewing Coordinates System* adalah:

$$P_{viewer} = M * \begin{bmatrix} x_w \\ y_w \\ 1 \end{bmatrix}$$

$$P_{viewer} = M * P_{word}$$

P_{word} = posisi titik terhadap *World Coordinates System*

P_{viewer} = posisi titik terhadap *Viewing Coordinates System*

Catatan:

Jika posisi *Viewing Coordinates System* berimpit dengan *World Coordinates System*, maka kita tidak perlu melakukan *Viewing Transformation*.

Contoh 5.1

Sebuah segitiga T dibentuk oleh titik-titik (3,1), (4,1) dan (4,2) terletak pada *World Coordinates System* dilihat oleh seorang pengamat dengan arah pandang atas $P - P_0$. Arah pandang atas

ini yang dipakai sebagai *Viewing Coordinates System*. Hitunglah koordinat-koordinat segitiga T terhadap pengamat jika,

a) $P_0=(2,1)$ dan $P=(3.5,3)$

b) $P_0=(0,0)$ dan $P=(0,3)$

Jawab:

a) $P=(3.5,3)$, $P_0=(2,1)$

Vektor satuan v :

$$v = \frac{P - P_0}{|P - P_0|} = \frac{(3.5, 3) - (2, 1)}{\sqrt{(1.5)^2 + (2)^2}} = \frac{(1.5, 2)}{\sqrt{6.25}} = (0.6, 0.8)$$

Vektor satuan u :

$$u = (v_y, -v_x) = (0.8, -0.6)$$

Matriks transformasi M :

$$M = \begin{bmatrix} u_x & u_y & 0 \\ v_x & v_y & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -x_0 \\ 0 & 1 & -y_0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.8 & -0.6 & 0 \\ 0.6 & 0.8 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -2 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix}$$

$$M = \begin{bmatrix} 0.8 & -0.6 & -1.0 \\ 0.6 & 0.8 & -2.2 \\ 0 & 0 & 1 \end{bmatrix}$$

$$P_{viewer} = M * P_{word}$$

$$P_{viewer} = \begin{bmatrix} 0.8 & -0.6 & -1 \\ 0.6 & 0.8 & -2.2 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 3 & 4 & 4 \\ 1 & 1 & 2 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0.8 & 1 & 1.6 \\ 0.6 & 0.2 & 1.2 \\ 1 & 1 & 1 \end{bmatrix}$$

Koordinat-koordinat segitiga T, terhadap pengamat adalah $(0.8, 0.6)$, $(1, 2)$, $(1.6, 1.2)$

b) $P=(0,0)$, $P_0=(0,3)$

$$v = \frac{P - P_0}{|P - P_0|} = \frac{(0, 0) - (0, 3)}{|(0, 0) - (0, 3)|} = \frac{(0, -3)}{\sqrt{3^2}} = (0, -1)$$

$$u = (v_y, -v_x) = (1, 0)$$

$$M = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

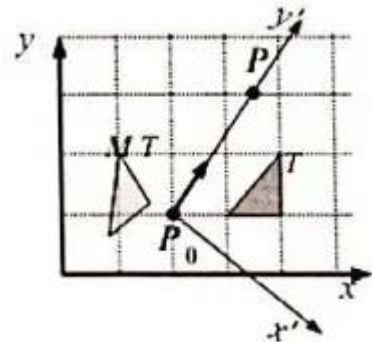
$$P_{viewer} = M * P_{word}$$

$$P_{viewer} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 3 & 4 & 4 \\ 1 & 1 & 2 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 4 & 4 \\ 1 & 1 & 2 \\ 1 & 1 & 1 \end{bmatrix}$$

Koordinat-koordinat segitiga T, terhadap pengamat adalah sama yaitu: $(3, 1)$, $(4, 1)$ dan $(4, 2)$. Ini artinya segitiga tersebut bila dilihat dari *World Coordinates System*, nilainya sama dengan ketika dilihat dari *Viewing Coordinates System*.

Tampak dari contoh 5.1(b), bahwasanya koordinat pengamat (*Viewing Coordinates System*) tepat berimpit dengan *World Coordinates System*, hal ini ditunjukkan oleh vektor satuan berikut:

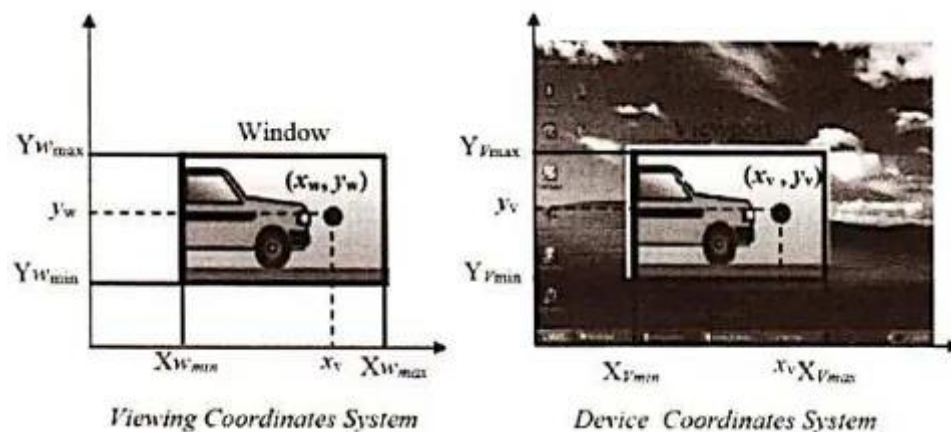
u $(1, 0)$ dan v $(0, 1)$, masing-masing menunjukan sumbu x dan sumbu y dari *world coordinates*



system. Sehingga titik-titik yang didapat dari *world coordinates system* sama persis dengan titik-titik yang dilihat dari *viewing coordinates system*.

5.1.3 Viewport Transformation

Perhatikan gambar 5.5. *Window* merupakan daerah yang dibatasi segi empat pada *Viewing Coordinates System* atau *World Coordinates System*, sedangkan *Viewport* adalah bagian dari layar dimana gambar yang ditangkap oleh *window* ditampilkan di *Device Coordinates System* (dilayar). Jadi *window* memilih bagian gambar yang akan ditampilkan dilayar; sedangkan *viewport* menunjukkan di mana posisi bagian gambar tersebut ditampilkan di layar.



(Gambar yang menunjukkan proses transformasi dari window ke viewport)

Gambar 5.5 Transformasi dari window ke viewport

Ada dua cara untuk menghitung transformasi dari *Viewing Coordinates System* ke *Device Coordinates System*, yaitu *Viewport Transformation* tanpa normalisasi dan dengan normalisasi.

A. Viewport Transformation tanpa Normalisasi

Gambar 5.5 menunjukkan sebuah titik yang terletak di (x_w, y_w) pada *window* ditransformasi ke (x_v, y_v) pada koordinat *viewport* dengan persamaan berikut:

$$\frac{x_w - x_{w_{\min}}}{x_{w_{\max}} - x_{w_{\min}}} = \frac{x_v - x_{v_{\min}}}{x_{v_{\max}} - x_{v_{\min}}} \quad \text{dan} \quad \frac{y_w - y_{w_{\min}}}{y_{w_{\max}} - y_{w_{\min}}} = \frac{y_v - y_{v_{\min}}}{y_{v_{\max}} - y_{v_{\min}}}$$

Atau

$$x_v = S_x x_w + t_x$$

$$y_v = S_y y_w + t_y$$

Di mana

$$S_x = \frac{x_{v_{\max}} - x_{v_{\min}}}{x_{w_{\max}} - x_{w_{\min}}}, \quad S_y = \frac{y_{v_{\max}} - y_{v_{\min}}}{y_{w_{\max}} - y_{w_{\min}}}$$

Dan

$$t_x = \frac{x_{w_{\max}} x_{v_{\min}} - x_{w_{\min}} x_{v_{\max}}}{x_{w_{\max}} - x_{w_{\min}}}, \quad t_y = \frac{y_{w_{\max}} y_{v_{\min}} - y_{w_{\min}} y_{v_{\max}}}{y_{w_{\max}} - y_{w_{\min}}}$$

Dalam bentuk matriks dapat ditulis sebagai:

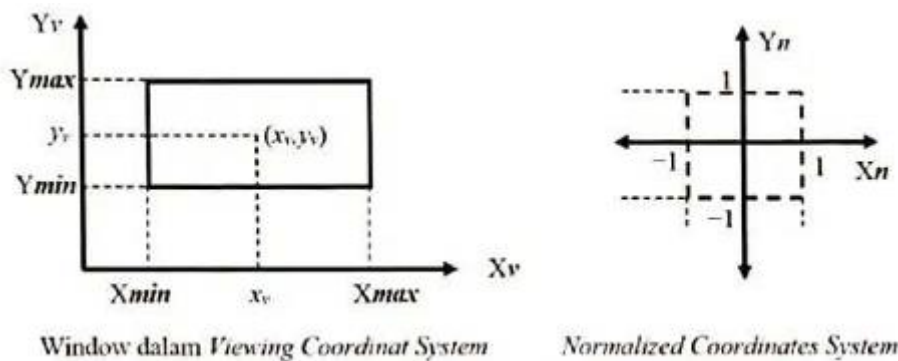
$$\begin{bmatrix} x_v \\ y_v \\ 1 \end{bmatrix} = \begin{bmatrix} S_x & 0 & t_x \\ 0 & S_y & t_y \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x_w \\ y_w \\ 1 \end{bmatrix}$$

B. Viewport Transformation menggunakan Normalisasi

Pada umumnya ukuran *device coordinates* berbeda-beda tergantung dari kemampuan *graphics card*. Berikut adalah beberapa contoh ukuran *device coordinates*.

Standar	x-maksimal	y-maksimal	Jumlah keseluruhan piksel
VGA	640	480	307.200
SVGA	800	600	480.000
XGA	1024	768	786.432
SXGA	1280	1024	1.228.800

Karena alasan inilah maka ada beberapa sistem grafik yang menggunakan *Normalized Coordinates*, yaitu koordinat dinormalisasi ke dalam range $[-1,1]$ seperti gambar 5.6, agar nantinya bila terjadi transformasi dari *Viewing Coordinates System* ke *Device Coordinates System* tidak terpengaruh dengan ukuran *Graphics Card*. Teknik ini diperlukan untuk menjaga proporsionalitas ukuran objek.



Gambar 5.6 Normalisasi koordinat

Sehingga, transformasi dari *Viewing Coordinate System* ke *Normalized Coordinates System* adalah:

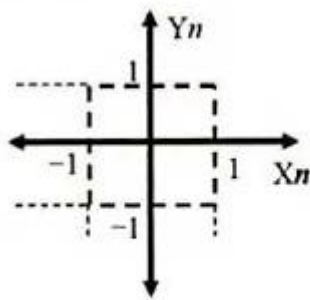
$$\frac{x_n - (-1)}{1 - (-1)} = \frac{x_v - xv_{\min}}{xv_{\max} - xv_{\min}} \text{ atau } x_n = -1 + \frac{2}{xv_{\max} - xv_{\min}} (x_v - xv_{\min})$$

$$\frac{y_n - (-1)}{1 - (-1)} = \frac{y_v - yv_{\min}}{yv_{\max} - yv_{\min}} \text{ atau } y_n = -1 + \frac{2}{yv_{\max} - yv_{\min}} (y_v - yv_{\min})$$

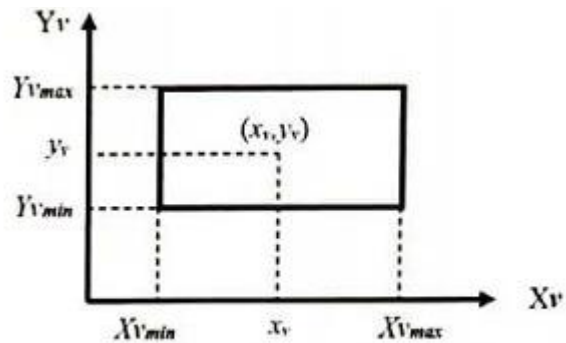
Dalam bentuk matriks:

$$\begin{bmatrix} x_n \\ y_n \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{2}{xv_{\max} - xv_{\min}} & 0 & -\frac{xv_{\max} + xv_{\min}}{xv_{\max} - xv_{\min}} \\ 0 & \frac{2}{yv_{\max} - yv_{\min}} & -\frac{yv_{\max} + yv_{\min}}{yv_{\max} - yv_{\min}} \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x_v \\ y_v \\ 1 \end{bmatrix}$$

Setelah dilakukan normalisasi, proses berikutnya adalah transformasi dari *Normalized Coordinates* ke *Viewport (Device Coordinates)*.



Normalized Coordinates System



Device Coordinates System

$$\frac{x_n - (-1)}{1 - (-1)} = \frac{x_v - xv_{\min}}{xv_{\max} - xv_{\min}} \text{ atau } x_v = xv_{\min} + \frac{(xv_{\max} - xv_{\min})}{2}(x_n + 1)$$

$$\frac{y_n - (-1)}{1 - (-1)} = \frac{y_v - yv_{\min}}{yv_{\max} - yv_{\min}} \text{ atau } y_v = yv_{\min} + \frac{(yv_{\max} - yv_{\min})}{2}(y_n + 1)$$

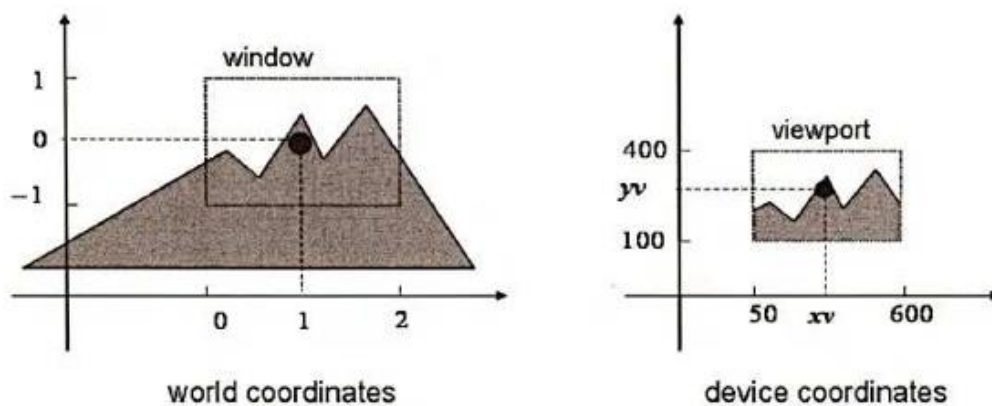
Dalam bentuk matriks:

$$\begin{bmatrix} x_v \\ y_v \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{(xv_{\max} - xv_{\min})}{2} & 0 & \frac{(xv_{\max} + xv_{\min})}{2} \\ 0 & \frac{(yv_{\max} - yv_{\min})}{2} & \frac{(yv_{\max} + yv_{\min})}{2} \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x_n \\ y_n \\ 1 \end{bmatrix}$$

Contoh 5.2:

Diketahui sebuah titik terletak di (1, 0) pada *World Coordinates System* dilihat melalui sebuah *window* berukuran (0, -1) – (2, 1). Tentukan posisi titik tersebut pada *Device Coordinates*, bila titik tersebut ditempatkan pada *viewport* berukuran (50, 100) – (600, 400) seperti pada Gambar 5-7:

- tanpa *Normalized Coordinates*,
- dengan *Normalized Coordinates*
- dengan *Normalized Coordinates* menggunakan standar VGA 640 × 480
- dengan *Normalized Coordinates* menggunakan standar SVGA 800 × 600



Gambar 5.7 Sebuah titik terletak di (1,0) pada *World Coordinates System*

Jawab:

Dalam hal ini *Viewing Coordinates System* berimpit dengan *World Coordinates System*, sehingga kita tidak perlu melakukan *viewing transformation*. Jadi perhitungan bisa dilakukan dari *World Coordinates System* ke *Viewport Coordinates System*.

(a) Tanpa *Normalized Coordinates*

$$\frac{x_w - xw_{\min}}{xw_{\max} - xw_{\min}} = \frac{x_v - xv_{\min}}{xv_{\max} - xv_{\min}} \qquad \frac{y_w - yw_{\min}}{yw_{\max} - yw_{\min}} = \frac{y_v - yv_{\min}}{yv_{\max} - yv_{\min}}$$

$$\frac{1-0}{2-0} = \frac{x_v - 50}{600-50} \qquad \frac{0-(-1)}{1-(-1)} = \frac{y_v - 100}{400-100}$$

$$x_v = 325 \qquad y_v = 250$$

(b) Dengan *Normalized Coordinates*

$$x_n = -1 + \frac{2}{xw_{\max} - xw_{\min}}(x_w - xw_{\min}) \qquad y_n = -1 + \frac{2}{yw_{\max} - yw_{\min}}(y_w - yw_{\min})$$

$$x_n = -1 + \frac{2}{2-0}(1-0) = 0 \qquad y_n = -1 + \frac{2}{1-(-1)}[0-(-1)] = 0$$

transformasi dari *Normalized Coordinates* ke *Viewport (Device Coordinates)*.

$$x_v = xv_{\min} + \frac{(xv_{\max} - xv_{\min})}{2}(x_n + 1) \qquad y_v = yv_{\min} + \frac{(yv_{\max} - yv_{\min})}{2}(y_n + 1)$$

$$x_v = 50 + \frac{(600-50)}{2}(0+1) = 325 \qquad y_v = 100 + \frac{(400-100)}{2}(0+1) = 250$$

(c) dengan *Normalized Coordinates* menggunakan standar VGA 640 × 480 \

$$x_v = xv_{\min} + \frac{(xv_{\max} - xv_{\min})}{2}(x_n + 1) \qquad y_v = yv_{\min} + \frac{(yv_{\max} - yv_{\min})}{2}(y_n + 1)$$

$$x_v = 0 + \frac{(640-0)}{2}(0+1) = 320 \qquad y_v = 0 + \frac{(480-0)}{2}(0+1) = 240$$

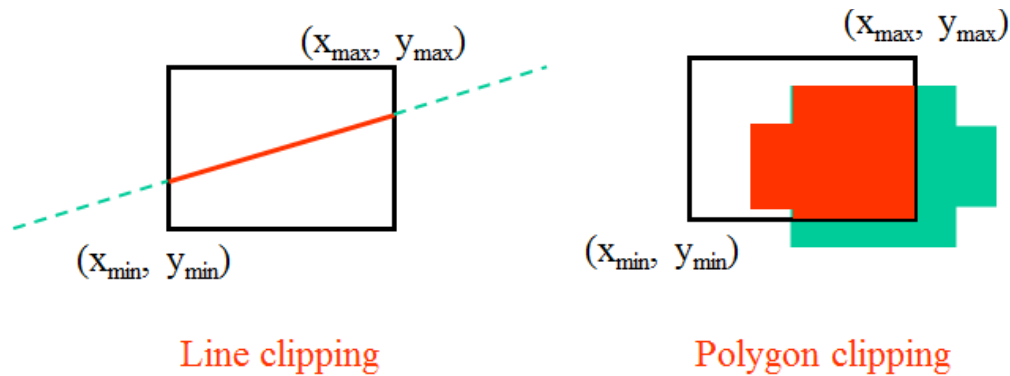
(d) dengan *Normalized Coordinates* menggunakan standar SVGA 800 × 600

$$x_v = xv_{\min} + \frac{(xv_{\max} - xv_{\min})}{2}(x_n + 1) \qquad y_v = yv_{\min} + \frac{(yv_{\max} - yv_{\min})}{2}(y_n + 1)$$

$$x_v = 0 + \frac{(800-0)}{2}(0+1) = 400 \qquad y_v = 0 + \frac{(600-0)}{2}(0+1) = 300$$

5.2 Clipping

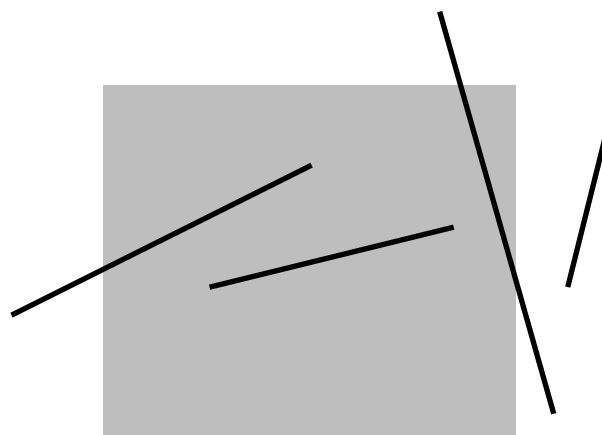
Clipping adalah proses pemotongan objek atau pengguntingan objek sehingga hanya objek yang berada pada area yang menjadi perhatian saja yang terlihat. Proses ini merupakan hal yang bisa dengan teknologi yang ada dewasa ini, namun proses *internal* pemrograman di dalamnya tidak sesederhana memakainya. Gambar di bawah ini mengilustrasikan proses *clipping* garis dan *clipping* poligon.



Gambar 5.8 Line Clipping dan Polygon Clipping

Garis-garis yang tampak pada area gambar atau *viewport* dapat dikelompokkan menjadi tiga yaitu:

1. Garis yang terlihat seluruhnya (*Fully visible*)
2. Garis yang hanya terlihat sebagian (*Partiality Visible*)
3. Garis yang tidak terlihat sama sekali (*Fully Invisible*)

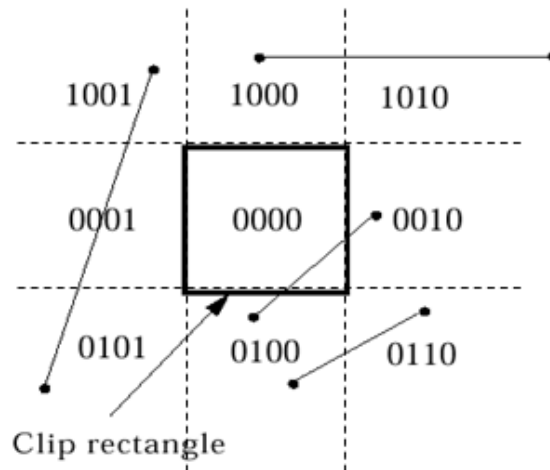


Gambar 5.9 Konsep Garis dalam Viewport

Proses *clipping* dilakukan terhadap garis-garis yang berada pada kondisi ke-2 yaitu garis-garis yang hanya terlihat sebagian saja.

5.2.1 Clipping Garis Cohen-Sutherland

Pada algoritma Cohen-Sutherland, *region viewport* dibagi menjadi 9 dan masing-masing memiliki kode *bit* atau *bit code* yang terdiri dari 4 *bit* yang menyatakan kondisi dari garis yang melalui *viewport* atau *region* yang dimaksud.



Gambar 5.10 Bit Code dalam Cohen-Sutherland

Kode empat *bit* menunjukkan posisi ujung garis pada *region viewport*.

0	0	0	0
Top (atas)	Bottom (bawah)	Right (kanan)	Left (kiri)

Sebagai contoh garis dengan ujung yang memiliki kode **bit 1001** artinya ujungnya ada di kiri atas *viewport*. Ujung dengan kode **bit 0010** berarti ada di kanan *viewport*. Dengan mengacu kepada kode *bit* maka proses *clipping* akan dilakukan secara lebih mudah dan efisien.

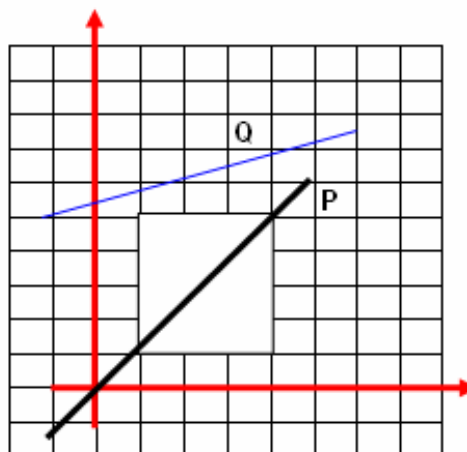
Contoh:

Diketahui koordinat *viewport* atau *area* gambar adalah (X_{min} , Y_{min} , X_{max} , Y_{max}) yaitu (1, 1, 4, 5) dan dua buah garis *P* dengan koordinat (-1, 2), (5, 6) dan garis *Q* dengan koordinat (-1, 5), (6, 7).

- Tentukan *region code* atau kode *bit* dari garis-garis tersebut
- Apabila statusnya *partially visible*, tentukan titik potongnya dengan batas *viewport*

Jawab:

Permasalahan dapat divisualkan pada gambar berikut.



Garis P:

Ujung garis P (-1, -2)

L = 1; karena $x < x_{Min}$ atau $-1 < 1$

R = 0; karena $x < x_{Max}$ atau $-1 < 4$

B = 1; karena $y < y_{Min}$ atau $-2 < 1$

T = 0; karena $y < y_{Max}$ atau $-2 < 5$

Dengan demikian region code untuk ujung P (-1,-2) adalah 0101

Ujung garis P (5, 6)

L = 0; karena $x > x_{Min}$ atau $5 > 1$

R = 1; karena $x > x_{Max}$ atau $5 > 4$

B = 0; karena $y > y_{Min}$ atau $6 > 1$

T = 1; karena $y > y_{Max}$ atau $6 > 5$

Dengan demikian region code untuk ujung P (5,6) adalah 1010

Karena region code dari kedua ujung garis tidak sama dengan 0000 maka garis P bersifat *partially invisible* dan perlu dipotong.

Garis Q:

Dengan cara yang sama pada garis P maka akan ditentukan *region code*: Ujung garis Q (-1,5) mempunyai region code = 0001

Ujung garis Q (6,7) mempunyai region code = 1010

Karena *region code* tidak sama dengan 0000 maka garis Q bersifat kemungkinan *partially invisible* dan perlu dipotong.

Penentuan Titik Potong dengan Region

Dilakukan berdasarkan tabel berikut.

Region Bit	Berpotongan Dengan	Dicari	Titik Potong
L=1	x_{Min}	yP1	(x_{Min} , yP1)
R=1	x_{Max}	yP2	(x_{Max} , yP2)
B=1	y_{Min}	xP1	(xP1, y_{Min})
T=1	y_{Max}	xP2	(xP2, y_{Max})

x_{P1} , x_{P2} , y_{P1} dan y_{P2} dihitung dengan formulasi berikut ini.

$$x_{P1} = x_1 + \frac{y_{Min} - y_1}{M}$$

$$x_{P2} = x_1 + \frac{y_{Max} - y_1}{M}$$

$$y_{P1} = y_1 + M * (x_{Min} - x_1)$$

$$y_{P2} = y_1 + M * (x_{Max} - x_1)$$

Dimana M dihitung dengan formula

$$M = \frac{Y2 - Y1}{X2 - X1}$$

Titik potong P (-1,-2), (5,6)

$$M = \frac{6 - (-2)}{5 - (-1)} = \frac{8}{6}$$

Region code di (-1,2) adalah 0101 (TBRL), berarti B=1 dan L=1

$$L=1 \text{ berarti } yP1 = y1 + M * (X \text{ min} - x1) = -2 + \frac{8}{6} * (1 - (-1)) = \frac{2}{3} = 0.86$$

Titik potongnya adalah (1, 0.86).

$$B=1 \text{ berarti } xP1 = x1 + \frac{yMin - y1}{M} = -1 + \frac{1 - (-2)}{\frac{8}{6}} = 1.25$$

Titik potongnya adalah (1.25, 1).

Region code di (5,6) adalah 1010 (TBRL), berarti T=1 dan R=1

$$R=1 \text{ berarti } yP2 = y1 + M * (xMax - x1) = 6 + \frac{8}{6} * (4 - 5) = 4.7$$

Titik potongnya adalah (4, 4.7)

$$T=1 \text{ berarti } xP2 = x1 + \frac{yMax - y1}{M} = 5 + \frac{5 - 6}{\frac{8}{6}} = 4.25$$

Titik potongnya adalah (4.25, 5)

Ada titik potong pada garis P yaitu (1, 0.67), (1.25, 1), (4, 4.7), (4.25, 5).

Pilih titik potong yang terdapat dalam *viewport* yaitu (1.25, 1) dan (4, 4.7).

5.2.2 Clipping Garis Liang-Barsky

Liang-Barsky menemukan algoritma *clipping* garis yang lebih cepat. *Clipping* yang lebih cepat dikembangkan berdasarkan persamaan parametrik dari segmen garis dapat ditulis dalam bentuk:

$$x = x1 + u \cdot dx$$

$$y = y1 + u \cdot dy$$

Dimana $dx = x2 - x1$ dan $dy = y2 - y1$. Dimana nilai $u \in [0, 1]$. Menurut Liang dan Barsky bentuk pertidaksamaan sebagai berikut:

$$xwmin \leq x1 + u \cdot dx \leq xwmax$$

$$ywmin \leq y1 + u \cdot dy \leq ywmax$$

Dengan:

$$u \cdot pk \leq qk,$$

$$k = 1, 2, 3, 4$$

Dimana parameter p dan q ditentukan sebagai berikut:

$$k = 1 \text{ (Kiri): } p1 = -dx, q1 = x1 - xwmin$$

$$k = 2 \text{ (Kanan): } p2 = dx, q2 = xwmax - x1$$

$$k = 3 \text{ (Bawah): } p3 = -dy, q3 = y1 - ywmin$$

$k = 4$ (Atas): $p_4 = dy$, $q_4 = y_{wmax} - y_1$

Garis yang sejajar dengan salah satu batas *clipping* mempunyai $p_k = 0$ untuk nilai $k=1,2,3,4$ yaitu *left*, *right*, *bottom*, dan *top*. Untuk setiap nilai k , juga diperoleh $q_k < 0$, maka garis sepenuhnya diluar batas *clipping*. Bila $q_k \geq 0$, maka garis didalam dan sejajar batas *clipping*. Bila $p_k < 0$, garis memotong batas *clipping* dari luar ke dalam, dan bila $p_k > 0$, garis memotong batas *clipping* dari dalam ke luar. Untuk nilai p_k yang tidak sama dengan 0, nilai u dapat diperoleh dengan

$$u = q_k / p_k$$

Untuk setiap garis, dapat dihitung nilai dan parameter u_1 dan u_2 yang menentukan posisi garis dalam bidang *clipping*. Nilai u_1 diperlihatkan dengan batas *clipping* dimana garis memotong batas *clipping* dari luar ke dalam ($p < 0$).

$$r_k = q_k / p_k$$

Dengan nilai u_1 adalah nilai maksimum dari nilai 0 dan bermacam-macam nilai r . Sebaliknya nilai u_2 ditentukan dengan memeriksa batas dimana *clipping* dipotong oleh garis dari dalam keluar ($p > 0$). nilai r_k dihitung untuk setiap batas *clipping*, dan nilai u_2 merupakan nilai minimum dari sekumpulan nilai yang terdiri dari 1 dan nilai r yang dihasilkan. Bila $u_1 > u_2$, maka garis sepenuhnya berada di luar *clip window* dan dapat dihilangkan; sebaliknya bila tidak ujung dari garis yang di clip dihitung dari dua nilai parameter u .

Untuk ($p_i < 0$) $t_1 = \text{Max}(r_k)$

Untuk ($p_i > 0$) $t_2 = \text{Min}(r_k)$

Jika $t_1 < t_2$ cari nilai ujung yang baru.

$t_1 \rightarrow (x_1 + dx * t_1, y_1 + dy * t_1)$ titik awal garis yang baru

$t_2 \rightarrow (x_1 + dx * t_1, y_1 + dy * t_1)$ titik ujung garis yang baru

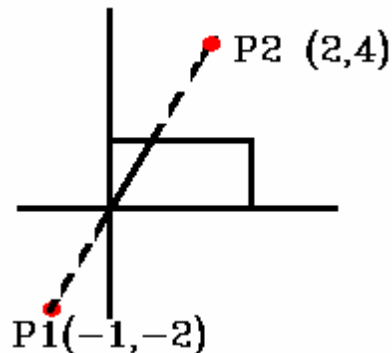
Algoritma Liang-Barsky lebih efisien dibandingkan dengan Cohen-Sutherland karena perhitungan titik potong dihilangkan.

Algoritma adalah sebagai berikut:

1. Initialize line intersection paramter: $m_1 \leftarrow 0$, $m_2 \leftarrow 1$
2. Compute dx , dy
3. For each window boundray
 - Repeat
 - Compute q, p
 - If $p < 0$ (outside > inside) Then
 - Compute $\text{Int} = q / p$
 - If $\text{Int} > m_2$ Then reject
 - Else If $\text{Int} > m_1$ Then $m_1 \leftarrow \text{Int}$
 - Else If $p > 0$ (inside > outside) Then
 - Compute $\text{Int} \leftarrow q/p$
 - If $\text{Int} < m_1$ Then Reject
 - Else If $\text{Int} < m_2$ Then $m_2 \leftarrow \text{Int}$ Else $p = 0$
 - If $q < 0$ reject
4. If m_1 is greater Than 0 (has been modified) compute new x_1, y_1
5. If m_2 is less Than 1 (has been modified) compute new x_2, y_2

Contoh:

Diketahui titik garis P1(-1,-2) dan P2(2,4) dan viewport $x_l=0$, $x_r=1$, $y_b=0$ dan $y_t=1$. Tentukan endpoint baru.

**Jawab:**

P1(-1,-2) , P2(2,4)

$x_L=0$, $x_R=1$, $y_B=0$, $y_T=1$

$$dx = x_2 - x_1$$

$$= 2 - (-1) = 3$$

$$p_1 = -dx,$$

$$= -3$$

$$P_2 = dx,$$

$$= 3$$

$$P_3 = -dy,$$

$$= -6$$

$$P_4 = dy,$$

$$= 6$$

$$dy = y_2 - y_1$$

$$= 4 - (-2) = 6$$

$$q_1 = x_1 - x_L$$

$$= -1 - 0 = -1$$

$$q_2 = x_R - x_1$$

$$= 1 - (-1)$$

$$q_3 = y_1 - y_B$$

$$= -2 - 0 = -2$$

$$q_4 = y_T - y_1$$

$$= 1 - (-2) = 3$$

$$\rightarrow q_1/p_1 = -1/-3$$

$$= 1/3$$

$$\rightarrow q_2/p_2 = 2/3$$

$$= 2$$

$$\rightarrow q_3/p_3 = -2/-6$$

$$= 1/3$$

$$\rightarrow q_4/p_4 = 3/6$$

$$= 1/2$$

Untuk ($p_i < 0$) $T_1 = \text{"Max"} (1/3, 1/3, 0) = 1/3$

Untuk ($p_i > 0$) $T_2 = \text{Min} (2/3, 1/2, 1) = 1/2$

$T_1 < T_2$

Perhitungan ujung baru

$$T_1 = 1/3$$

$$X_1' = x_1 + dx * t_1$$

$$= -1 + (3 * 1/3) = -1 + 1 = 0$$

$$Y_1' = y_1 + dy * t_1$$

$$= -2 + 6 * 1/3 = -2 + 2 = 0$$

$$(X_1', Y_1') = (0, 0)$$

$$T_2 = 1/2$$

$$X_2' = x_1 + dx * t_2$$

$$= -1 + (3 * 1/2) = 1/2$$

$$Y_2' = y_1 + dy * t_2$$

$$= -2 + 6 * 1/2 = 1$$

$$(X_2', Y_2') = (1/2, 1)$$

5.3 Implementasi clipping garis dengan Java

A. Kode lengkap clipping garis dengan algoritma cohen-sutherland

```
// 9-region code constants (same as classic)
final int INSIDE = 0;    // 0000
final int LEFT = 1;     // 0001
final int RIGHT = 2;    // 0010
final int BOTTOM = 4;   // 0100
final int TOP = 8;      // 1000

// Clipping rectangle
float xmin = 100, ymin = 100;
float xmax = 300, ymax = 300;

// Original line endpoints
float x1 = 50, y1 = 350;
float x2 = 350, y2 = 50;

void setup() {
  size(400, 400);
  background(255);
  strokeWeight(2);

  // Draw clipping window
  stroke(0);
  noFill();
  rect(xmin, ymin, xmax - xmin, ymax - ymin);

  // Draw original line in red
  stroke(255, 0, 0);
  line(x1, y1, x2, y2);

  // Perform Cohen-Sutherland Clipping with 9-region logic
  float[] clipped = cohenSutherlandClip(x1, y1, x2, y2);

  if (clipped != null) {
    // Draw clipped line in green
    stroke(0, 200, 0);
    line(clipped[0], clipped[1], clipped[2], clipped[3]);
  }
}

// Compute 9-region code for point (x, y)
int computeRegionCode(float x, float y) {
  int code = INSIDE;
  if (x < xmin) code |= LEFT;
  else if (x > xmax) code |= RIGHT;
  if (y < ymin) code |= BOTTOM;
  else if (y > ymax) code |= TOP;
  return code;
}

// Cohen-Sutherland Clipping Algorithm using 9-region code
float[] cohenSutherlandClip(float x1, float y1, float x2, float y2) {
  int code1 = computeRegionCode(x1, y1);
  int code2 = computeRegionCode(x2, y2);
  boolean accept = false;

  while (true) {
    if ((code1 | code2) == 0) {

```

```

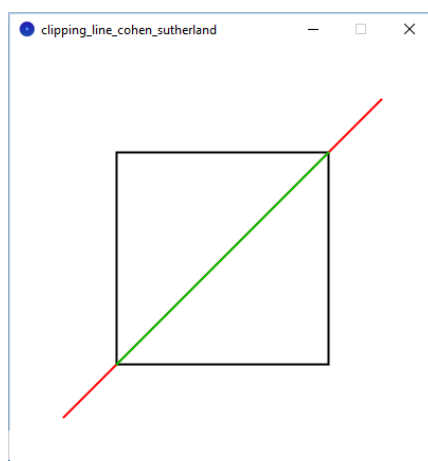
// Trivial accept
accept = true;
break;
} else if ((code1 & code2) != 0) {
// Trivial reject
break;
} else {
// At least one endpoint is outside
float x = 0, y = 0;
int outCode = (code1 != 0) ? code1 : code2;

if ((outCode & TOP) != 0) {
x = x1 + (x2 - x1) * (ymax - y1) / (y2 - y1);
y = ymax;
} else if ((outCode & BOTTOM) != 0) {
x = x1 + (x2 - x1) * (ymin - y1) / (y2 - y1);
y = ymin;
} else if ((outCode & RIGHT) != 0) {
y = y1 + (y2 - y1) * (xmax - x1) / (x2 - x1);
x = xmax;
} else if ((outCode & LEFT) != 0) {
y = y1 + (y2 - y1) * (xmin - x1) / (x2 - x1);
x = xmin;
}

if (outCode == code1) {
x1 = x;
y1 = y;
code1 = computeRegionCode(x1, y1);
} else {
x2 = x;
y2 = y;
code2 = computeRegionCode(x2, y2);
}
}

if (accept) {
return new float[]{x1, y1, x2, y2};
} else {
return null; // Rejected
}
}

```



B. Kode lengkap clipping garis dengan algoritma liang-barsky

```
// Clipping window boundaries
float xmin = 100, ymin = 100;
float xmax = 300, ymax = 300;

// Original line coordinates
float x0 = 50, y0 = 350;
float x1 = 350, y1 = 50;

void setup() {
  size(400, 400);
  background(255);
  strokeWeight(2);

  // Draw clipping window
  stroke(0);
  noFill();
  rect(xmin, ymin, xmax - xmin, ymax - ymin);

  // Draw original line (red)
  stroke(255, 0, 0);
  line(x0, y0, x1, y1);

  // Liang-Barsky Clipping
  float[] clipped = liangBarsky(xmin, xmax, ymin, ymax, x0, y0, x1, y1);
  if (clipped != null) {
    // Draw clipped line (green)
    stroke(0, 200, 0);
    line(clipped[0], clipped[1], clipped[2], clipped[3]);
  }
}

// Liang-Barsky algorithm function
float[] liangBarsky(float xmin, float xmax, float ymin, float ymax,
float x0, float y0, float x1, float y1) {
  float dx = x1 - x0;
  float dy = y1 - y0;
  float u1 = 0.0f, u2 = 1.0f;
  float[] p = {-dx, dx, -dy, dy};
  float[] q = {x0 - xmin, xmax - x0, y0 - ymin, ymax - y0};

  for (int i = 0; i < 4; i++) {
    if (p[i] == 0) {
      if (q[i] < 0) {
        return null; // Line is parallel and outside
      }
    } else {
      float r = q[i] / p[i];
      if (p[i] < 0) {
        u1 = max(u1, r); // entering
      } else {
        u2 = min(u2, r); // leaving
      }
    }
  }

  if (u1 > u2) return null; // Line is outside the clip window

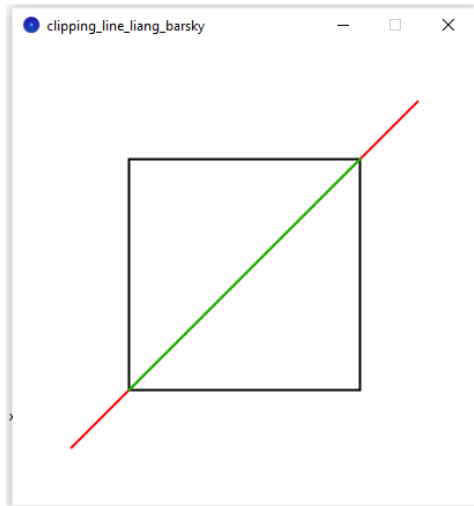
  // Calculate clipped line endpoints
  float cx0 = x0 + u1 * dx;
```

```

float cy0 = y0 + u1 * dy;
float cx1 = x0 + u2 * dx;
float cy1 = y0 + u2 * dy;

return new float[]{cx0, cy0, cx1, cy1};
}

```



5.4 Soal Latihan

1. Diketahui titik awal P (1,1) dan titik akhir di Q (10,10), dengan area clipping xmin=1, ymin=1, xmax=7 dan ymax=7. Selesaikan masalah ini dengan clipping Cohen-Sutherland.
2. Berdasarkan soal nomor 1 lakukan clipping menggunakan algoritma Liang-Barsky dimana xl=1, xr= 7, yb=1 dan yt =7.

Daftar Pustaka

1. Andono, Pulung Nurtantio., Sutojo, T. 2016. Konsep Grafika Komputer. Yogyakarta: CV Andi Offset.
2. David F. Rogers, Alan J. Adams, Mathematical Elements for Computer Graphics (2nd edition), McGraw-Hill, 1989.
3. Edward Angel. 2005. *Interactive Computer Graphics: A Top-Down Approach with OpenGL 2nd*. Addison Wesley.
4. Grafika Komputer untuk Mahasiswa dan Umum. Universitas Negeri Malang.
5. John F. Hughes, Andries Van Dam, Morgan Mcguire, David F. Sklar, James D. Foley, Steven K. Feiner, Kurt Akeley. 2014. *Computer Graphics: Principles and Practice (3rd edition)*. Addison-Wesley.
6. Klawonn, Frank. 2012. *Introduction to Computer Graphics Using Java 2D and 3D 2nd Edition*. Springer-Verlag: London.
7. Maliki, Irfan. 2006. Grafika Komputer. Jakarta Selatan: Mediatek.
8. Siahaan, Vivian., Sianipar, Rismon H., Java untuk Grafika Komputer dan Animasi, SPARTA, 2018.